

Webクライアント

HTML読み込み後の動作

```
145: document.addEventListener('DOMContentLoaded', () => {
146:     const addBtn = document.getElementById('add_freq_btn');
147:     const freqListBody = document.getElementById('eis_freq_list');
148:
149:     addBtn.addEventListener('click', () => {
150:         const newRow = document.createElement('tr');
151:         newRow.innerHTML = `
152:             <td><input type="text" name="eis_freq" value=""></td>
153:             <td><input type="text" name="eis_avg_count" value=""></td>
154:             <td><button type="button" class="remove_freq_btn">x</button></td>
155:         `;
156:         freqListBody.appendChild(newRow);
157:     });
158:
159:     freqListBody.addEventListener('click', (event) => {
160:         if (event.target.classList.contains('remove_freq_btn')) {
161:             const rowToRemove = event.target.closest('tr');
162:             if (rowToRemove) {
163:                 rowToRemove.remove();
164:             }
165:         }
166:     });
167:
168:     // 校正データの読み込み
169:     const cal_ranges = [0, 100, 1000, 10000];
170:     cal_ranges.forEach(range => {
171:         const offset = localStorage.getItem(`cal_offset_${range}`);
172:         const coefficient = localStorage.getItem(`cal_coefficient_${range}`);
173:         const date = localStorage.getItem(`cal_date_${range}`);
174:         if (offset && coefficient) {
175:             calication_offset[range] = Number(offset);
176:             calication_coefficient[range] = Number(coefficient);
177:             if (range === 0 && calCheckboxAuto.checked) {
178:                 calStatusAuto.innerText = `自動: ${date}`;
179:             } else if (range === 100 && calCheckbox100.checked) {
180:                 calStatus100.innerText = `±15.15mA: ${date}`;
181:             } else if (range === 1000 && calCheckbox1000.checked) {
182:                 calStatus1000.innerText = `±1.65mA: ${date}`;
183:             } else if (range === 10000 && calCheckbox10000.checked) {
184:                 calStatus10000.innerText = `±150μA: ${date}`;
185:             }
186:         }
187:     });
188: });
```

146行から166行までで、EIS測定時の周波数や平均回数を設定する要素を作っている。これらはボタンのクリックによって要素を増やしたり減らしたりする必要があるため、HTML読み込み後にjavascriptから作っている。

校正データの読み込みについて

168行目からは、必要に応じてブラウザに保存された校正データを読み込んでいる。校正データはLocalStorageという領域に保存されている。LocalStorageに保存されているデータはブラウザを閉じたとしても半永久的に保持される。LocalStorageや、今回は使用していないが、他の保存領域については以下のサイトを参考にしてほしい。

https://qiita.com/hazure_engineer/items/68cc9b97c3d5f72567ef

CSVファイルの保存

```
213: CSVButton.addEventListener("click", () => {
214:     saveCSV(CSVTextbox.value)
215: });
216:
217: function saveCSV(filename) {
218:     const bom = new Uint8Array([0xef, 0xbb, 0xbf]);
219:     let data_csvText;
220:     if (dataType === "IVcurve") {
221:         data_csvText = ListToCSV(drawDataList, ["電圧(V)", "電流(mA)", "走査方向"]);
222:     } else if (dataType === "EIS") {
223:         data_csvText = ListToCSV(drawDataList, ["時間(s)", "出力電流(mA)", "入力電圧(V)", "測定時間(s)", "サンプリング周波数(Hz)", "平均回数"]);
224:     }
225:     const blob = new Blob([bom, data_csvText], { type: "text/csv" });
226:
227:     const link = document.createElement('a');
228:     link.download = filename;
229:     link.href = URL.createObjectURL(blob);
230:     link.click();
231:     URL.revokeObjectURL(link.href);
232:
233:     console.log(`CSVファイル出力完了。ファイル名:${filename}`)
234: }
235: function ListToCSV(list, header) {
236:     result = "";
237:     for(let i in header) {
238:         result = `${result}${header[i]},`;
239:     }
240:     result = result.slice(0, -1) + "\n";
241:     for(let i in list) {
242:         for(key in list[i]) {
243:             result = `${result}${list[i][key]},`;
244:         }
245:         result = result.slice(0, -1) + "\n";
246:     }
```

```
247:     return result;
248: };
```

測定データをダウンロードしたい需要はあるだろう。そんな時に便利なのがCSVという形式のファイルだ。CSVファイルには表データを保存することができ、Excelなどで開くことができる。例えば、以下のような表データがあったとする。

電圧	電流
0.1	5
0.2	6
0.3	7

これをCSVという形式で表すと、

```
電圧,電流
0.1,5
0.2,6
0.3,7
```

というようになる。つまり、カンマ(,)で列を区切り、改行で行を区切っているということだ。ちなみに列ラベル(今回なら電圧,電流)を区別する情報はCSVファイルに保存されない。読み込む際に手動で「1行目は列ラベルですよ」と伝える必要がある。

ではこれをjavascriptで生成してみる。まず、生成元の2次元リストが与えられる。そして、そのリストは列ラベル(プログラム内ではheaderと呼ばれている)と共に235行目から248行目にある`ListToCSV()`という関数に渡される。`ListToCSV()`はリストの中身を順番に取り出し、カンマと改行を付け加え、その結果を返している。

例えば、先程のCSVファイルを生成する場合は、

```
list = [[0.1, 5], [0.2, 6], [0.3, 7]]
header = ["電圧", "電流"]

csv = ListToCSV(list, header)
```

という具合になる。

あとは出来たファイルを自動でダウンロードさせれば良い。それを227行目から231行目で行っている。

[!NOTE] BOMについて

218行目と225行目に謎の処理がある。これは生成したCSVファイルにBOMと呼ばれる情報を追加している。これにより、このファイルのエンコード方法が指定されるため、外部ソフトで読み込む際の文字化けを防ぐことができる。

IVカーブ測定について

測定開始部分

```
252: function onIVcurveButtonClick() {
253:     stopStatusUpdate(-1);
254:     MODE = "IVcurve";
255:     const speed = Number(IV_speedTextbox.value) / 1000;
256:     let before_waiting_time;
257:     if (IV_counter == 0) {
258:         before_waiting_time = Number(IV_before_waitingTimeTextbox.value);
259:     }else {
260:         before_waiting_time = 0;
261:     }
262:     const stepVol = Number(IV_stepTextbox.value);
263:     const minVol = Number(IV_minTextbox.value);
264:     const maxVol = Number(IV_maxTextbox.value);
265:     const step_direction = IV_voltage_invertCheckbox.checked ? -1 : 1;
266:     const repetitions = Number(IV_repetitions.value);
267:     DAC_absVol = Number(DAC_maxVolTextbox.value) -
Number(DAC_minVolTextbox.value);
268:     DAC_absMinVol = Math.abs(Number(DAC_minVolTextbox.value));
269:     DAC_step = Number(DAC_stepTextbox.value);
270:     IV_R = Number(IV_resistance.value);
271:     const ADC_IconvR = Number(ADC_IconvRTextbox.value);
272:     const ADC_IconvR_time = Number(ADC_IconvRtimeTextbox.value);
273:     const measure_count = Number(IV_measure_countTextbox.value);
274:
275:     // この関数では使わないが、データ取得時のために更新しておく
276:     ADC_absVol = Number(ADC_maxVolTextbox.value) -
Number(ADC_minVolTextbox.value);
277:     ADC_absMinVol = Math.abs(Number(ADC_minVolTextbox.value));
278:     ADC_step = Number(ADC_stepTextbox.value);
279:
280:     const waiting_time = Number(IV_waitingTimeTextbox.value);
281:     const invert = IV_invertCheckbox.checked ? 1 : 0;
282:     ADC_invert = ADC_invertMeasCheckbox.checked ? -1 : 1;
283:
284:     const step_speed = Math.round((speed / DAC_absVol) * DAC_step);
285:     const step_stepVol = Math.round((stepVol / DAC_absVol) * DAC_step);
286:     const step_minVol = Math.round(((minVol + DAC_absMinVol) / DAC_absVol) *
DAC_step);
287:     const step_maxVol = Math.round(((maxVol + DAC_absMinVol) / DAC_absVol) *
DAC_step);
288:
289:     if (IV_counter == 0) {
290:         IV_counter = measure_count
291:     }
292:
293:     status_text.innerHTML = `IVカーブ測定中...<br>( 現在の測定進捗:
${measure_count - IV_counter + 1}回目/${measure_count}回 )`;
294:     ButtonEnd("IVcurve_start");
295:
296:     console.log(`IVcurve 0 0 ${step_speed} ${step_stepVol} ${waiting_time}
${step_minVol} ${step_maxVol} ${ADC_IconvR} ${ADC_IconvR_time} ${repetitions}
${invert} ${step_direction} ${before_waiting_time}`);
```

```

297:     writeTextSerial(`IVcurve 0 0 ${step_speed} ${step_stepVol}
${waiting_time} ${step_minVol} ${step_maxVol} ${ADC_IconvR} ${ADC_IconvR_time}
${repetitions} ${invert} ${step_direction} ${before_waiting_time}`);
298: }

```

253行目の`stopStatusUpdate(-1)`により、測定中の状態更新を止めている。その後、255行目から273行目、280行目から287行目でマイコン向けに値を変換している。

276行目から278行目では、測定データの変換に必要なパラメータを取得している。257行目から261行目、289行目から291行目にある処理は複数回測定を行うための処理である。例えば、測定回数が5に設定された場合は最初に`IV_counter`が5に設定され、測定後に`IV_counter`が-1される。その後、自動で再度測定が行われ、これが`IV_counter`が0になるまで繰り返される。257行目から261行目にある場合分けは、2回目以降の測定では測定前の待機時間が必要無いという処理をしている。

複数回測定は、サイクリックボルタンメトリー(CV)測定を行う際に必要となる。CV測定については以下を参照してほしい。

<https://www.bas.co.jp/1118.html> \

[!NOTE] 三項演算子について

281行目、282行目で使われているのが三項演算子という書き方だ。ちょっとしたif文を一行で書ける便利な書き方である。プログラムの行数が減るというメリットはあるが、可読性が下がるので適材適所という感じ。詳細は以下を参照してほしい。

<https://qiita.com/yasu-hoshi/items/2f88e725ea81ac402bd2>

測定データ処理部分

```

667:     if (MODE === "IVcurve") {
668:         if (noCtrlCharText === "START") {
669:             console.log("記録開始...")
670:             recording = true;
671:             drawDataList = [];
672:             dataType = "none";
673:             createGraph();
674:         }else if (noCtrlCharText === "END") {
675:             recording = false;
676:             console.log("記録終了...")
677:             status_text.innerText = "接続完了";
678:             MODE = "NORMAL";
679:             ButtonEndi("IVcurve_finish");
680:
681:             if (IV_counter > 1) {
682:                 IV_counter--;
683:                 onIVcurveButtonClick();
684:             }else {
685:                 IV_counter = 0;
686:                 startStatusUpdate();
687:             }
688:         }else if (recording) {
689:             const rawInputStepVol = noCtrlCharText.split(" ")[0];
690:             const rawOutputStepVol = noCtrlCharText.split(" ")[1];
691:             const outputStepVol = (rawOutputStepVol - (ADC_absMinVol /

```

```
ADC_absVol) * ADC_step) * ADC_invert;
692:         let I_conversion_R = noCtrlCharText.split(" ")[2];
693:         const INV = noCtrlCharText.split(" ")[3];
694:
695:         // 電流電圧変換における並列抵抗を考慮した補正
696:         if (I_conversion_R == 100) {
697:             I_conversion_R = 99.009900990099;
698:         } else if (I_conversion_R == 1000) {
699:             I_conversion_R = 909.09090909091;
700:         }
701:
702:         // 安定用の直列抵抗を考慮した補正
703:         const current = ((outputStepVol / ADC_step) * ADC_absVol) /
I_conversion_R;
704:         let converted_voltage = (rawInputStepVol / DAC_step) * DAC_absVol
- DAC_absMinVol;
705:         converted_voltage = converted_voltage - IV_R * current;
706:
707:         // 校正の適用
708:         let calibrated_current;
709:         if (calCheckbox.checked) {
710:             isCalibrated = true;
711:             let offset = 0;
712:             let coefficient = 1;
713:
714:             // まず、現在のレンジ専用の校正データを探す
715:             // なければ自動レンジの校正データを使う
716:             if (noCtrlCharText.split(" ")[2] === "100") {
717:                 if (calCheckbox100.checked === true && ("100" in
calication_coefficient)) {
718:                     offset = calication_offset["100"];
719:                     coefficient = calication_coefficient["100"];
720:                     //console.log("100レンジの校正データを使用");
721:                 } else if ("0" in calication_coefficient &&
calCheckboxAuto.checked === true) {
722:                     offset = calication_offset["0"];
723:                     coefficient = calication_coefficient["0"];
724:                     //console.log("自動レンジの校正データを使用");
725:                 }
726:             } else if (noCtrlCharText.split(" ")[2] === "1000") {
727:                 if (calCheckbox1000.checked === true && ("1000" in
calication_coefficient)) {
728:                     offset = calication_offset["1000"];
729:                     coefficient = calication_coefficient["1000"];
730:                     //console.log("1000レンジの校正データを使用");
731:                 } else if ("0" in calication_coefficient &&
calCheckboxAuto.checked === true) {
732:                     offset = calication_offset["0"];
733:                     coefficient = calication_coefficient["0"];
734:                     //console.log("自動レンジの校正データを使用");
735:                 }
736:             } else if (noCtrlCharText.split(" ")[2] === "10000") {
737:                 if (calCheckbox10000.checked === true && ("10000" in
calication_coefficient)) {
```

```

738:             offset = calication_offset["10000"];
739:             coefficient = calication_coefficient["10000"];
740:             //console.log("10000レンジの校正データを使用");
741:         }else if ("0" in calication_coefficient &&
calCheckboxAuto.checked === true) {
742:             offset = calication_offset["0"];
743:             coefficient = calication_coefficient["0"];
744:             //console.log("自動レンジの校正データを使用");
745:         }
746:     }
747:
748:     calibrated_current = (current * 1000 * coefficient) + offset;
749: }else {
750:     isCalibrated = false;
751:     calibrated_current = current * 1000;
752: }
753:
754:     graph.data.datasets[INV].data.push({x: converted_voltage, y:
calibrated_current});
755:     graph.update();
756:     drawDataList.push({
757:         "x": converted_voltage,
758:         "y": calibrated_current,
759:         "INV": INV
760:     });
761:     dataType = "IVcurve";
762: }else {
763:     parseSerial(text);
764: }

```

667行目から673行目は測定データを受信する準備を行っている。IVカーブ測定の場合は測定データがリアルタイムに送信されるため、この段階でグラフも作成している。(ただし、中身のデータは無いため空白のグラフである)

送信順に則り、次は688行目から761行目を見ていく。ここでは送られてくるデータを処理している。696行目から752行目は測定値の補正を行っている。その後、754行目から755行目で測定値をグラフに反映し、ファイル保存・校正用の`drawDataList`にも測定値を入れている。

かつては`drawDataList`に測定データを全て入れた後、`drawDataList`をもとにグラフを作成していたため、ファイル保存・校正用にも関わらずこのような名前になっている。深い理由はない。最後に674行目から687行目を見ていく。ここでは測定データの受信を止める処理を行っている。また、681行目から683行目では複数回測定を行う場合の処理、684行目から686行目では止めていた状態更新を再開させている。

EIS測定について

```

302: async function onEISButtonClick() {
303:     DAC_absVol = Number(DAC_maxVolTextbox.value) -
Number(DAC_minVolTextbox.value);
304:     DAC_absMinVol = Math.abs(Number(DAC_minVolTextbox.value));
305:     DAC_step = Number(DAC_stepTextbox.value);

```



```
306:     const ADC_IconvR = Number(ADC_IconvRTextbox.value);
307:
308:     if (ADC_IconvR == 0) {
309:         sendSerialConsole("EIS測定を行う場合、自動レンジは使用できません。「ADC電流レン  
ジ:」から自動レンジ以外を選択してください。", "red");
310:         return
311:     }
312:
313:     stopStatusUpdate(-1);
314:     getAllFreq();
315:
316:     const mesure_times = EIS_freqs.length;
317:     const amp_voltage = Number(EISampText.value) / 1000;
318:     const offset_voltage = Number(EISoffsetVolText.value) / 1000;
319:     const high_voltage = offset_voltage + amp_voltage / 2;
320:     const low_voltage = offset_voltage - amp_voltage / 2;
321:     const step_high_voltage = Math.round(((high_voltage + DAC_absMinVol) /  
DAC_absVol) * DAC_step);
322:     const step_low_voltage = Math.round(((low_voltage + DAC_absMinVol) /  
DAC_absVol) * DAC_step);
323:
324:     // この関数では使わないが、データ取得時のために更新しておく
325:     ADC_absVol = Number(ADC_maxVolTextbox.value) -  
Number(ADC_minVolTextbox.value);
326:     ADC_absMinVol = Math.abs(Number(ADC_minVolTextbox.value));
327:     ADC_step = Number(ADC_stepTextbox.value);
328:     ADC_invert = ADC_invertMeasCheckbox.checked ? -1 : 1;
329:
330:     let input_delay_time;
331:     let sampfreq;
332:     let input_freq;
333:
334:     ButtonEnDi("IVcurve_start");
335:     for (let i = 0; i < mesure_times; i++) {
336:         input_freq = EIS_freqs[i].freq;
337:         input_delay_time = Math.round(((1 / input_freq) * 1000 / 2)*100)/100;
338:
339:         sampfreq = EIS_max_data * input_freq;
340:         let closestN = 1;
341:         let minDiff = Infinity;
342:         let sampfreq_arrange = 0;
343:         // RPPicoは自然数μs単位でしか動作できないため、もっとも切りの良いサンプリングレートを探  
す。
344:         for (let n = 1; n <= 100000; n++) {
345:             let candidateRate = 1000000 / n;
346:             let diff = Math.abs(candidateRate - sampfreq);
347:             if (diff < minDiff) {
348:                 minDiff = diff;
349:                 closestN = n;
350:                 sampfreq_arrange = candidateRate;
351:             } else {
352:                 // 差が増加し始めたら、最適値を超えたと判断して打ち切る
353:                 break;
354:             }
```



```

355:         }
356:         sampfreq_arrange = Math.round(Math.min(sampfreq_arrange,
EIS_max_sampling_freq) * 100) / 100
357:         EIS_time = 1/input_freq;
358:         EIS_sampling = sampfreq_arrange;
359:         EIS_avg = EIS_freqs[i].avg_count;
360:         EIS_voltageList = [];
361:
362:         EIS_avgMode = false;
363:         for (let i = 0; i < EIS_avg; i++) {
364:             status_text.innerText = `EIS測定中...(${i+1}/${measure_times}) 平均
回数:${i+1}/${EIS_avg}回`;
365:             EIS_avgCountNow = i;
366:             MODE = "EIS";
367:             writeTextSerial(`EIS 0 0 ${sampfreq_arrange} ${input_delay_time}
${step_low_voltage} ${step_high_voltage} ${ADC_IconvR} 1
${DEBUG_HARDWARE_AVERAGE_COUNT}`);
368:             const flagPromise = new Promise(resolve => {
369:                 EIS_flag_promise = resolve;
370:             });
371:             // EISの計測が終わるまで待機する
372:             await flagPromise;
373:
374:             EIS_finished_flag = false;
375:             EIS_avgMode = true;
376:         }
377:         EIS_avgMode = false;
378:         if (EISsaveBox.checked) {
379:             let filename = CSVTextbox.value.replace(".csv", "");
380:             let freq_str = String(input_freq).replace(".", "-");
381:             filename = `${filename}_${freq_str}Hz.csv`;
382:             saveCSV(filename);
383:         }
384:     }
385:     startStatusUpdate();
386:     status_text.innerText = "接続完了";
387: }

```

IVカーブ測定と似た処理が多いため、プログラムの説明は省く。ただ、実行順序が変わっている部分があるため、それに関してはフローチャートを参照してほしい。 \

測定データ処理部分

```

765:     }else if (MODE === "EIS") {
766:         if (noCtrlCharText === "START") {
767:             console.log("記録開始...")
768:             drawDataList = [];
769:             recording = true;
770:             EIS_measure_count = 0;
771:         }else if (noCtrlCharText === "END") {
772:             recording = false;

```

```
773:         console.log("記録終了...")
774:
775:         if (graph) {
776:             graph.destroy();
777:             console.log("destroyed");
778:         }
779:         drawGraph(drawDataList);
780:         ButtonEndi("IVcurve_finish");
781:         if (EIS_flag_promise) {
782:             EIS_flag_promise();
783:         }
784:     }else if (noCtrlCharText.split(" ")[1] === "error:") {
785:         parseSerial(text);
786:         recording = false;
787:         if (graph) {
788:             graph.destroy();
789:             console.log("destroyed");
790:         }
791:         drawGraph(drawDataList);
792:         ButtonEndi("IVcurve_finish");
793:         if (EIS_flag_promise) {
794:             EIS_flag_promise();
795:         }
796:     }else if (recording) {
797:         const time_count = noCtrlCharText.split(" ")[0];
798:         const raw_DAC_vol_step = noCtrlCharText.split(" ")[1];
799:         const raw_ADC_vol_step = noCtrlCharText.split(" ")[2] /
DEBUG_HARDWARE_AVERAGE_COUNT;
800:         const ADC_vol_step = (raw_ADC_vol_step - (ADC_absMinVol /
ADC_absVol) * ADC_step) * ADC_invert;
801:         let I_conversion_R = noCtrlCharText.split(" ")[3];
802:
803:         // 電流電圧変換における並列抵抗を考慮した補正
804:         if (I_conversion_R == 100) {
805:             I_conversion_R = 99.009900990099;
806:         }else if (I_conversion_R == 1000) {
807:             I_conversion_R = 909.09090909091;
808:         }
809:
810:         const current = ((ADC_vol_step / ADC_step) * ADC_absVol) /
I_conversion_R;
811:         const voltage = (raw_DAC_vol_step / DAC_step) * DAC_absVol -
DAC_absMinVol;
812:
813:         // 校正の適用
814:         let calibrated_current;
815:         if (calCheckbox.checked) {
816:             isCalibrated = true;
817:             let offset = 0;
818:             let coefficient = 1;
819:
820:             // まず、現在のレンジ専用の校正データを探す
821:             // なければ自動レンジの校正データを使う
822:             if (noCtrlCharText.split(" ")[2] === "100") {
```

```
823:             if (calCheckbox100.checked === true && ("100" in
calication_coefficient)) {
824:                 offset = calication_offset["100"];
825:                 coefficient = calication_coefficient["100"];
826:                 //console.log("100レンジの校正データを使用");
827:             }else if ("0" in calication_coefficient &&
calCheckboxAuto.checked === true) {
828:                 offset = calication_offset["0"];
829:                 coefficient = calication_coefficient["0"];
830:                 //console.log("自動レンジの校正データを使用");
831:             }
832:         }else if (noCtrlCharText.split(" ")[2] === "1000") {
833:             if (calCheckbox1000.checked === true && ("1000" in
calication_coefficient)) {
834:                 offset = calication_offset["1000"];
835:                 coefficient = calication_coefficient["1000"];
836:                 //console.log("1000レンジの校正データを使用");
837:             }else if ("0" in calication_coefficient &&
calCheckboxAuto.checked === true) {
838:                 offset = calication_offset["0"];
839:                 coefficient = calication_coefficient["0"];
840:                 //console.log("自動レンジの校正データを使用");
841:             }
842:         }else if (noCtrlCharText.split(" ")[2] === "10000") {
843:             if (calCheckbox10000.checked === true && ("10000" in
calication_coefficient)) {
844:                 offset = calication_offset["10000"];
845:                 coefficient = calication_coefficient["10000"];
846:                 //console.log("10000レンジの校正データを使用");
847:             }else if ("0" in calication_coefficient &&
calCheckboxAuto.checked === true) {
848:                 offset = calication_offset["0"];
849:                 coefficient = calication_coefficient["0"];
850:                 //console.log("自動レンジの校正データを使用");
851:             }
852:         }
853:
854:         calibrated_current = (current * 1000 * coefficient) + offset;
855:     }else {
856:         isCalibrated = false;
857:         calibrated_current = current * 1000;
858:     }
859:
860:     if (EIS_avgMode) {
861:         EIS_voltageList[EIS_measure_count] += calibrated_current;
862:         drawDataList[EIS_measure_count] = {
863:             "x": time_count / EIS_sampling,
864:             "y": EIS_voltageList[EIS_measure_count] /
(EIS_avgCountNow + 1),
865:             "INPvol": voltage,
866:             "measureTime": EIS_time,
867:             "samplingHz": EIS_sampling,
868:             "avgCount": EIS_avg
869:         };
```

```
870:         }else {
871:             drawDataList.push({
872:                 "x": time_count / EIS_sampling,
873:                 "y": calibrated_current,
874:                 "INPvol": voltage,
875:                 "measureTime": EIS_time,
876:                 "samplingHz": EIS_sampling,
877:                 "avgCount": EIS_avg
878:             });
879:             EIS_voltageList[EIS_measure_count] = calibrated_current;
880:         }
881:         dataType = "EIS";
882:         EIS_measure_count++;
883:     }else {
884:         parseSerial(text);
885:     }
```

766行目から770行目はIVカーブ測定とほとんど同じである。ただ、EIS測定では測定データがリアルタイムに届かないので、この段階ではグラフの作成を行っていない。

796行目から882行目では測定データの処理を行っている。こちらもIVカーブ測定と同じ部分が多いが、平均化の処理でいくつか違いがある。

平均化の処理について

そもそも、EIS測定における平均化には2種類ある。一つはハードウェア、つまりIMPACT側で行う平均化だ。もう一つはクライアント側で行う平均化だ。前者は各測定点において複数回測定を行い、平均化する方法。後者は測定完了後に同様の測定をもう一度行い、平均化する方法。ただ、言葉よりフローチャートを見たほうが分かりやすいと思う。

ハードウェア平均化では測定結果が加算された状態で渡されるので、799行目のように、最初に測定回数で割っている。クライアント平均化では860行目から869行目のように前回の測定データとの平均を都度計算している。

771行目から783行目では測定終了の処理をしている。IVカーブ測定と異なるのはグラフを作成していることと、782行目で測定の終了を通知していることだ。`EIS_flag_promise()`を実行することで初めて372行目以降のプログラムが実行される。また、784行目から795行目にもIVカーブ測定との違いがある。EIS測定を行う前に、メモリ容量的に測定が中止されることがある。この部分は測定中止時に適切に測定を終了させるための処理。

シリアル通信

[テクニック>Webクライアント>シリアル通信について](#) を参照

校正

[テクニック>Webクライアント>校正について](#) を参照

テクニック

ここではシリアル通信や校正についてなど、単なるプログラムの解説ではなくて幅広く活用できるであろう内容について解説する。

エラーが起きた時

正直、環境によるエラーも多く、一概にどう対応すれば良いと言えることはない。まず、エラーメッセージを読むのが大切だと思う。ただ、エラーメッセージをまるっとAIに投げて聞くのが早いし解決率も高い。もし、以下のプログラムでエラーが発生したらAIに聞いてほしい。

Webクライアント

同期処理

JavaScriptは待機が必要な処理に関しては処理の完了を待たずに次の処理を実行する非同期処理を行う。ただ、時には処理の完了を待ってほしい時もある。超簡単に言うと、関数の頭に`async`という文字列を付けて、あとは待機させたい処理の頭に`await`という文字列を付ければok。

```
1: async function longProcess() {
2:   console.log("処理開始!");
3:   await AnywayLongProcess();
4:   console.log("処理終了!");
5: }
```

ただ実際は、Promiseというのを使って、上のプログラムで言う`AnywayLongProcess()`を自分で実装したりすることもある。

詳細は以下の記事などを参考に。

<https://zenn.dev/singularity/articles/360d69fde8322d>

<https://qiita.com/MoriP-K/items/c9590934c7e72638107b>

シリアル通信について

Webブラウザ上でシリアル通信を行うには、Web Serial APIと呼ばれる仕組みを使う必要がある。互換性などについては以下のページにまとめられている。

https://developer.mozilla.org/ja/docs/Web/API/Web_Serial_API

注意しなければいけないのは、現時点(2026年3月23日時点)で完璧に動作するのはWindowsPCの特定ブラウザのみということだ。モバイルに関してはAndroidで一部サポートされているのみである。Apple系はPC、モバイル含めて全く動作しない。

もしiPad上でマイコンと通信したいのであればWeb Bluetooth APIとBluefyと呼ばれる専用ブラウザを使用するのが良いと思う。

https://developer.mozilla.org/ja/docs/Web/API/Web_Bluetooth_API

[!NOTE] Webブラウザ上で動作することについて

そもそも、Webブラウザ上でデバイスの機能(シリアルポートやBluetoothなど)を使うというコンセプト自体がまだまだ発展途上である。また、危険性の観点から制限も多い(シリアル通信でアップデートする機器を乗っ取ったりできてしまう)。Apple系で動作しないという話やモバイルでの制限もそういった背景がある。 \

実際の実装について

IMPACTで使っている処理を簡略化したものを使用する。また、以下の記事を参考にした。

<https://qiita.com/youtoy/items/6394a5570ddd54a2ce87>

<https://qiita.com/youtoy/items/f77dc205e817f31a4e18>

また、記事内の説明からプログラムに関しては一部、以下のサイトから引用したものがあると思う。

<https://developer.chrome.com/docs/capabilities/serial?hl=ja>

接続時の処理

```
01: async function onConnectButtonClick() {
02:   try {
03:     const filter = { usbVendorId: 0x2E8A };
04:     port = await navigator.serial.requestPort({ filters: [filter] });
05:     await port.open({ baudRate: 115200 });
06:     console.log("接続成功");
07:     readTextSerial();
08:   } catch (e) {
09:     if (e.name === 'NotFoundError') {
10:       console.error("NotFoundError: シリアルポートが選択されませんでした。シリアルポ-
11: トへの接続が許可されていない可能性があります。");
12:     } else if (e.name === 'NetworkError') {
13:       console.error("NetworkError: シリアルポートへの接続に失敗しました。別のアプリケ-
14: ーションがシリアルポートを使用している可能性があります。");
15:     } else if (e.name === 'InvalidStateError') {
16:       console.error("InvalidStateError: そのシリアルポートは既に開かれています。");
17:     } else {
18:       console.error(`不明なエラー: ${e.message}`);
19:     }
20:   }
21: }
```

まず、4行目のプログラムが実行された段階でユーザーに接続先を選択させるダイアログが表示される。

`port`は他の関数でも利用するため、関数の外でグローバル変数として定義している。もし、特定の機器に接続させたいのであれば、3行目のようにフィルターを設定することをおすすめする。今回の場合であれば、`usbVendorId`(機器の販売会社を識別するコード)がRaspberry Pi 財団を示す`0x2E8A`に設定されているため、Raspberrypi系のデバイスしかダイアログに表示されなくなる。

その後、5行目でユーザーが選択した機器に接続を試みる。`baudRate`というのは通信速度で、機器側と合わせる必要がある。

もし、3行目から7行目の間で何かエラーが発生すれば9行目以降の例外処理が実行される。

7行目の`readTextSerial()`に関しては次で説明する。

読み取り処理

```
01: class LineBreakTransformer {
02:   constructor() {
03:     this.chunks = "";
04:   }
05:
06:   transform(chunk, controller) {
```

```
07:     this.chunks += chunk;
08:     const lines = this.chunks.split("\r\n");
09:     this.chunks = lines.pop();
10:     lines.forEach((line) => controller.enqueue(line));
11:   }
12:
13:   flush(controller) {
14:     controller.enqueue(this.chunks);
15:   }
16: }
17:
18: async function readTextSerial() {
19:   let textDecoder = new TextDecoderStream();
20:   port.readable.pipeTo(textDecoder.writable);
21:   while (port.readable) {
22:     const reader = textDecoder.readable
23:       .pipeThrough(new TransformStream(new LineBreakTransformer()))
24:       .getReader();
25:     try {
26:       while (true) {
27:         const { value, done } = await reader.read();
28:         if (done) {
29:           console.log("Canceled");
30:           break;
31:         }
32:         console.log(value);
33:       }
34:     } catch (error) {
35:       console.error(error);
36:     } finally {
37:       reader.releaseLock();
38:     }
39:   }
40: }
```

まず、前提としてシリアル通信で送られてくるデータは0と1からなるバイト列である。そのため、バイト列を文字列に変換する必要がある。

バイト列(01010011....) -(変換)-> 文字列(Hello)

また、得られるデータは「チャンク」と呼ばれる塊で分割されている。例えば、

Hello World!\r\n (\r\nは改行を表す)

という文字列を受信したいという場合であっても、

Hello, World!\r\n

という2つの文字列に分割されてしまうかもしれないということである。ただ、それは不便であるから、できれば文字列は改行ごとに取得したい。そのため、受信した文字列は一時的に保存しておいて、\r\nが出てきたらそこで分割するという処理が必要である。

最終的な処理の流れは以下ようになる。

バイト列(01010011....) -(変換)-> 文字列(Hello) -(処理)-> 一文の文字列(Hello World!\r\n)

では実際の実装を見ていく。まず19行目でTextDecoderStreamと呼ばれる物を取得している。これはバイト列を文字列に変換してくれるものだ。これを20行目でシリアル通信の読み取り部分と接続している。これに

より、読み取ったバイト列は自動的にTextDecoderStreamに渡されるようになる。これで以下のようなデータの流れが出来上がる。

バイト列 -(変換)-> 文字列

23行目で出てくるLineBreakTransformerというのは1行目で定義しているように、文字列を一時的に保存して、改行ごとに分割してくれる処理である。これをTextDecoderStreamに接続している。これで以下のようなデータの流れが出来上がる。

バイト列 -(変換)-> 文字列 -(処理)-> 一文の文字列

あとは27行目のようにすれば一行ずつシリアル通信の読み取りを行うことができる。読み取った文に対して何か処理を行いたい場合は32行目の部分に入れれば良い。このプログラムでは単にデバッグコンソールに文を出力している。

書き込み処理

```
1: async function writeTextSerial(text) {
2:   const encoder = new TextEncoder();
3:   const writer = port.writable.getWriter();
4:   await writer.write(encoder.encode(text + "\n"));
5:   console.log("テキスト書き込み: " + text);
6:   writer.releaseLock();
7: }
```

書き込み処理は先程よりも単純である。なぜなら、与えられる文字列は必ず一文であるため、以下のような変換の流れを作るだけで良いからだ。 一文の文字列(Hello World!) -(変換)-> バイト列(01010011....)

校正について

例えば、正確な抵抗値の分かる抵抗などを測定機につなげ、以下のような測定値が得られた際に、(真の値に関しては抵抗値から算出する)

電圧値	測定値	真の値
0.1	3	1
0.2	5	2
0.3	7	3
0.4	9	4
0.5	11	5

測定値を真の値に近づける方法を考える。まず、一番実装が単純なのは測定値→真の値の変換表を作ることだ。

測定値→真の値の対応表を作る方法

測定結果と抵抗値をもとに以下のような対応表が完成する。

測定値	真の値
-----	-----

測定値	真の値
3	1
5	2
7	3
9	4
11	5

そして、別の抵抗を測定したときに以下のような測定値が得られた場合、上記の表をもとに校正を行うことができる。

電圧値	測定値	校正後の値
0.2	3	1
0.4	5	2
0.6	7	3
0.8	9	4
1.0	11	5

この手法は単に測定値と真の値の関係を保存しておくだけで良く、実装が単純である。また、非線形(特定の値以降でズレが生じているなど)の誤差も校正することができる。

一方で、校正時に測定されてない値に関しては校正を適用することができない(例えば、上記の場合であれば、測定値が1であれば校正できない)。また、校正時の測定値にノイズが乗っている場合、その影響も受けてしまう。

とはいえそれなりに万能なので、まずはこの手法で実装してみて、限界を感じたら他の手法に挑戦するのが良いと思う。

直線近似の傾きとy切片から求める方法

誤差の原因を考えると、大抵は抵抗値の誤差による傾きの変化とオフセットになる。例えば、「校正について」の最初に提示した表の場合であれば、真の値は $y=10x$ (y は電流値、 x は電圧値)で、測定値は $y=20x + 1$ になる。つまり、傾きは真の値の2倍、かつ+1オフセットしている(y 切片+1)ということになる。つまり、測定値から-1して、1/2すれば校正できるということになる。

例えば、別の抵抗を測定したときに以下のような測定値が得られた場合、

電圧値	測定値	校正後の値
0.2	3	1
0.4	5	2
0.6	7	3
0.8	9	4
1.0	11	5

このように校正できる。

より一般化すると、真の値が $y=ax + b$ 、測定値が $y = cx + d$ であるとき、測定値に $b - d$ を足した後、 a / c 倍すれば良いということである。

ただ、問題は a , b , c , d をどのように求めるかということだ。こんな時に便利なのが最小二乗法を用いた直線近似だ。これにより、最もフィットする直線の傾きや y 切片を求められる(直線と各測定点との差の二乗を最小化する)。

求め方は以下を参考にしてほしい。

<https://hooktail.sub.jp/computPhys/least-square/index.html>

この手法は校正時の測定値にノイズが乗っていてもあまり影響しない。その上、校正時に測定されていない値が測定されても、校正を適用することができる。

ただ、当然、傾きと値のオフセット以外の誤差は校正できない(以下のように)。

[!NOTE] RP2040(初代Raspberrypi Pico)のADCの誤差について

RP2040にはADCの値が特定の範囲で欠損してしまうハードウェア上のバグがある。以下のサイトを参考に

<http://radiopench.blog96.fc2.com/blog-entry-1189.html>

ファームウェア

まず前提として、C/C++とRaspberrypiPico, Pico2を使用した開発に絞って説明する。ただ、Pico,Pico2はMicroPythonやArduinoIDEを使用しても開発できる。それぞれの環境でどれくらいの性能が出るのかは以下の記事が参考になる。

<https://qiita.com/teddokano/items/a9e96a6b5c5142640f98> \

また、Pico、Pico2に関してはかなり詳細なデータシートが公開されている(一部、日本語版も公開されているが、バージョンが古いので英語版を読むことをおすすめする)。 \

データシート

CPUについて

メモリの動作や電源の動作など、CPUに関して書かれている。

RP2040(Pico):

<https://pip-assets.raspberrypi.com/categories/814-rp2040/documents/RP-008371-DS-1-rp2040-datasheet.pdf?disposition=inline>

RP2350(Pico2):

<https://pip-assets.raspberrypi.com/categories/1214-rp2350/documents/RP-008373-DS-2-rp2350-datasheet.pdf?disposition=inline>

Pico, Pico2ボードについて

公式が出しているマイコンボード、Pico, Pico2に関して書かれている。ボードの寸法や回路図が載っているのはもちろんのこと、設計意図なども一部書かれている。

Pico:

<https://pip-assets.raspberrypi.com/categories/610-raspberry-pi-pico/documents/RP-008307-DS-1-pico-datasheet.pdf?disposition=inline>

Pico2:

<https://pip-assets.raspberrypi.com/categories/1005-raspberry-pi-pico-2/documents/RP-008299-DS-2-pico-2-datasheet.pdf?disposition=inline>

C/C++ sdkについて

RaspberrypiPicoオリジナルのプログラムの動作や書き方が書かれている。ただ、種類によってはそこまで丁寧な説明が無いため、結局は動作を確認しながら書いていくことになる。

ネット:

<https://www.raspberrypi.com/documentation/pico-sdk/>

PDF:

<https://pip-assets.raspberrypi.com/categories/609-microcontroller-boards/documents/RP-009085-KB-1-raspberry-pi-pico-c-sdk.pdf?disposition=inline>

サンプルコードについて

Pico, Pico2のほぼあらゆる機能を動作させるサンプルコードが公開されている。

<https://github.com/raspberrypi/pico-examples>

ボードの自作について

なんと、RP2040などのCPU単体を購入して、それを動かすマイコンボード(プリント基板)を設計する方法が書かれている。

恐らくここまですることは無いと思うが、マイコンボードの設計に関しては非常に参考になる。

<https://pip-assets.raspberrypi.com/categories/814-rp2040/documents/RP-008279-DS-1-hardware-design-with-rp2040.pdf?disposition=inline>

オーバークロック

標準でシステムクロックはPicoが125MHz、Pico2が150MHzに設定されている。Arduinoよりは高いがESP32には及ばないくらいの値であるため、そこそこ高速だろう。

しかし、必要に応じてシステムクロックを手動で上げることも可能となっている。例えば、IMPACTの場合、Pico2を280MHzにオーバークロックして使用している。

当然、上げすぎると動作が不安定になることもある。ただ、手軽にすべての処理が高速になるため、諸刃の剣という感じである。 \

具体的な手法

`set_sys_clock_pll(f_vco, post_div1, post_div2)`というプログラムでシステムクロックを変更することができる。プログラムのどこに配置しても動くのかは調べていないが、main関数の一番最初に配置するのが無難だろう。

注意点として、`f_vco`がそのままシステムクロックになるわけではない。システムクロックを`f_sys`とすると、

`f_sys = f_vco / ( post_div1 * post_div2 )`

という式で求められる。ただ、`f_vco`は750MHz~1600MHz、`post_div1`, `post_div2`は1~7の整数値という制限がある。例えば、システムクロックを125MHzにしたい場合は、

`f_vco = 1500 * MHz, post_div1 = 6, post_div2 = 2`

になり、280MHzにしたい場合は、

`f_vco = 1120 * MHz, post_div1 = 2, post_div2 = 2`

となる。

まあ、それぞれのパラメータを決定するのはそれほど難しくないが、公式が、目的のシステムクロックを実現できる最適なパラメータを求められるpythonスクリプトを公開している。

https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2_common/hardware_clocks/scripts/vcocalc.py
それぞれのパラメータの制限に関してはrp2040データシートの229ページ、rp2350の576ページを参考にした。

シリアル通信

書き込み

`printf("Hello, world!\n");`で文字列を送信することができる。

読み取り

```
01: char command[128];
02: scanf("%s", command);
03: printf("Received command: %s\n", command);
```

これが基本的な読み取り方法である。これで、最初の空白までの内容を文字列として読み取ることができる。

サンプル

```
01: #include "pico/stdlib.h"
02: #include <stdio.h>
03:
04: int main() {
05:     stdio_init_all();
06:     char command[128];
07:     int argument;
08:
09:     while (true) {
10:         scanf("%s", command);
11:         scanf("%d", &argument);
12:         printf("Received command: %s\n", command);
13:         printf("Received argument: %d\n", argument);
14:         sleep_ms(100);
15:     }
16: }
```

このプログラムを実行しつつ、例えば、`test 1234`と送信すると、

```
01: Received command: test
02: Received argument: 1234
```

という文字列が返ってくる。

より高度なシリアル通信

接続状態別LEDインジケータ

開発をしていると、「マイコンが完全に停止している」のか、「シリアル通信が死んでいる」のか、「受信しているが、処理はされていない」のか判断したい時がある(いずれも「PCから送信しているが応答は無い」という状態)。

そこで、ボードに搭載されているLEDに、接続待機時は1秒おきに点滅、接続後は受信したら一瞬点灯、という動作をさせることにする。ただ、このような動作は標準機能では再現できない。そこで、標準機能を使用せず、自前で用意することにする。

```
01: #include <stdio.h>
02: #include "pico/stdlib.h"
03:
04: // USBシリアル関連
05: #include "tusb.h"
06: #include "pico/stdio_usb.h"
07:
08: static bool g_is_connected = false;
09:
10: // タイマー関連
11: struct repeating_timer LED_timer;
12:
13: #define LED_PIN 25
14:
15: bool LED_timer_callback(struct repeating_timer *t) {
16:     if (gpio_get(LED_PIN) == 1) {
17:         gpio_put(LED_PIN, 0);
18:     } else {
19:         gpio_put(LED_PIN, 1);
20:     }
21:     return true;
22: }
23:
24: int64_t LED_off_callback(alarm_id_t id, void *user_data) {
25:     gpio_put(LED_PIN, 0);
26:     return 0;
27: }
28: void tud_cdc_line_state_cb(uint8_t itf, bool dtr, bool rts) {
29:     (void) itf;
30:     (void) rts;
31:
32:     g_is_connected = dtr; // DTRがONなら接続、OFFなら切断
33:
34:     if (g_is_connected) {
35:         // 接続時は点滅を解除し、LEDを消灯
36:         cancel_repeating_timer(&LED_timer);
37:         gpio_put(LED_PIN, 0);
38:     } else {
```

```
39: // 接続待機時はLEDを1秒間隔で点滅
40: add_repeating_timer_ms(-1000, LED_timer_callback , NULL, &LED_timer);
41: gpio_put(LED_PIN, 1);
42: }
43: }
44:
45: int main() {
46:     stdio_init_all();
47:
48:     // USBシリアル初期化
49:     tusb_init();
50:     stdio_usb_init();
51:
52:     gpio_init(LED_PIN);
53:     gpio_set_dir(LED_PIN, GPIO_OUT);
54:     gpio_put(LED_PIN, 0);
55:
56:     char com_command[128];
57:
58:     printf("system started\n");
59:
60:     while (true) {
61:         tud_task();
62:
63:         if (g_is_connected) {
64:             int result = scanf("%s", &com_command);
65:
66:             if (result == 1) {
67:                 gpio_put(LED_PIN, 1);
68:                 add_alarm_in_ms(20, LED_off_callback, NULL, false);
69:                 printf("Received command: %s\n", com_command);
70:             }
71:         }
72:         else {
73:             sleep_ms(100);
74:         }
75:     }
76: }
```

15行目の関数は呼び出されるとLEDの明滅状態を反転させる。24行目の関数はLEDを消灯する。28行目の関数はどこからも呼び出されていない用に見えるが、実は内部で、usbの状態が変化したら呼び出されるようになっている。67行目と68行目が、シリアル通信を受信するとLEDを20ミリ秒だけ点灯させている。

`add_alarm_in_ms`は指定ミリ秒後に指定関数を実行してくれるものだ。よく見る実装方法だと、

```
01: gpio_put(LED_PIN, 1);
02: sleep_ms(20);
03: LED_off_callback();
```

という方法が思いつくかもしれない。ただ、この場合、20ミリ秒間CPUを占有してしまうため、その後の処理にも20ミリ秒の遅延が発生してしまう。一方、タイマ割り込みであればCPUを占有しない。指定時間が来

ていなければ先に以降のプログラムを実行してくれる。`add_repeating_timer_ms`もタイマ割り込みだ。もし、「〇秒後や〇秒おきに特定の処理を行いたい」かつ「別の処理も並行して行いたい」という場合があればタイマ割り込みを使うことをおすすめする。

シリアル通信経由での書き込みモード化

Pico, Pico2はBOOTSELボタンを押しながらUSBを抜き差しすると書き込みモードになる(リセットボタンがある場合はBOOTSELボタンを押しながらリセットボタンを押せば良い)。ただ、プログラムを書き込みたいたびにUSBを抜き差しするのも面倒くさい。

そんなときはシリアル通信経由で書き込みモードに切り替える方法がある。やり方は簡単で、ボーレート(通信速度)を1200にした状態でPico, Pico2に接続するだけ。すると、瞬時に書き込みモードになる。

ただ、当然、シリアル通信が死んでいる場合はこの方法は使えない。

シリアル通信上のデバイス名を変更

通常、Pico, Pico2のシリアル通信上の名前は「Board CDC」に固定されている。ただ、変えたい時もあるかもしれない。そんなときは内部で使われているプログラムを強引に書き換えることで名前を書き換えることができる。

その前に、一応、変更予定のプログラムを見ておく。

[https://github.com/raspberrypi/pico-](https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2_common/pico_stdio_usb/stdio_usb_descriptors.c)

[sdk/blob/master/src/rp2_common/pico_stdio_usb/stdio_usb_descriptors.c](https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2_common/pico_stdio_usb/stdio_usb_descriptors.c)

名前に関係する部分だけを抜粋すると、\

```
136: static const char *const usbd_desc_str[] = {
137:     [USB_STR_MANUF] = USB_MANUFACTURER,
138:     [USB_STR_PRODUCT] = USB_PRODUCT,
139:     [USB_STR_SERIAL] = usbd_serial_str,
140:     [USB_STR_CDC] = "Board CDC",
141: #if PICO_STDIO_USB_ENABLE_RESET_VIA_VENDOR_INTERFACE
142:     [USB_STR_RPI_RESET] = "Reset",
143: #endif
144: };
```

`USB_STR_MANUF`などは変更できるようになっているが、`USB_STR_CDC`だけがハードコードされている。これを以下のように書き換え、\

```
140: static const char *const usbd_desc_str[] = {
141:     [USB_STR_MANUF] = USB_MANUFACTURER,
142:     [USB_STR_PRODUCT] = USB_PRODUCT,
143:     [USB_STR_SERIAL] = usbd_serial_str,
144:     [USB_STR_CDC] = USB_CDC_PRODUCT_NAME,
145: #if PICO_STDIO_USB_ENABLE_RESET_VIA_VENDOR_INTERFACE
146:     [USB_STR_RPI_RESET] = "Reset",
147: #endif
148: };
```

また、`USBDCDC_PRODUCT_NAME`が指定されなかった時にデフォルト名を使用するようにする。\\

```
56: #ifndef USBDCDC_PRODUCT_NAME
57: #define USBDCDC_PRODUCT_NAME "Board CDC"
58: #endif
```

その後、この書き換えたプログラムを使用するように指定すると、名前が自由に変更できるようになる。ただ、このプログラムだけではなく、他の依存関係にあるプログラムもいくつか持ってこなければならぬ。

といったことを全てまとめたフォルダを以下に置いている。

https://github.com/oosan1/OriginalSMU_KKHS/tree/main/microcomputer/C_C%2B%2B/SMUcontrol/custom_src

これをメインのプログラムと同じディレクトリに配置し、`CMakeLists.txt`に以下のような記述を書き足せば良い。`SMUcontrol`や`SMUcontrol.c`の部分は適宜書き換えてほしい。

```
42: add_executable(SMUcontrol
43:     SMUcontrol.c
44:     custom_src/reset_interface.c
45:     custom_src/stdio_usb.c
46:     custom_src/stdio_usb_descriptors.c
47: )
48:
49: # USBシリアルの商品名等を定義
50: target_compile_definitions(SMUcontrol PRIVATE
51:     USBDCDC_MANUFACTURER="KKHS ScienceClub"
52:     USBDCDC_PRODUCT="SMU"
53:     USBDCDC_PRODUCT_NAME="IMPACT Interface"
54: )
~~~~~
70: target_include_directories(SMUcontrol PRIVATE
71:     ${CMAKE_CURRENT_LIST_DIR}
72:     ${CMAKE_CURRENT_LIST_DIR}/custom_src
73:     ${CMAKE_CURRENT_LIST_DIR}/custom_src/include
74: )
```